PROJETO PARA AVALIAÇÃO DAS DISCIPLINAS: TESTE E MANUTENÇÃO DE SISTEMAS
DATA INÍCIO: 27/01/2026
DATA DE ENTREGA: 02/03/2026

Objetivo do projeto: Desenvolver, **de forma individual**, um sistema web utilizando Django ou Laravel, aplicando conceitos de **Teste de Sistemas** e **Manutenção de Sistemas**. O projeto deverá contemplar funcionalidades essenciais de um sistema real, ser acompanhado por documentação técnica padronizada (plano de teste, casos de teste, relatórios e plano de manutenção) e passar por validação prática, onde cada aluno realizará testes no sistema de outro colega. O objetivo é garantir que os alunos pratiquem o ciclo completo de desenvolvimento, testes e manutenção, promovendo autonomia, responsabilidade e qualidade no código entregue.

1. Escolha de Cenário

Cada aluno deve escolher um sistema diferente em Django ou Laravel. Sugestões de cenários:

- Sistema de **Chamados de TI**
- Sistema de **Reservas Online** (hotel ou restaurante)
- Sistema de **Gestão de Cursos**
- Sistema de **Controle de Consultas Médicas**
- Sistema de **Pedidos Online** (delivery simples)
- Sistema de **Vendas de Automóveis**

1.1 Ações Mínimas Obrigatórias do Projeto

A - Autenticação e Segurança

- [ ] Sistema de login e logout.
- [ ] Recuperação de senha.
- [ ] Proteção contra acessos não autorizados.

B - Perfis de Acesso

- [ ] Diferentes níveis de usuário (ex.: administrador, usuário comum).
- [ ] Controle de permissões (quem pode criar, editar, excluir).

C - CRUD (Create, Read, Update, Delete)

- [ ] Pelo menos **uma entidade principal** com CRUD completo (ex.: clientes, reservas, chamados).
- [ ] Validação de dados nos formulários.
- [ ] Feedback claro ao usuário (mensagens de sucesso/erro).

D - Dashboard / Relatórios

- [ ] Tela inicial com visão geral do sistema.
- [ ] Relatórios básicos (ex.: quantidade de registros, status, métricas).
- [ ] Gráficos ou tabelas para visualização dos dados.

2. Troca de Projetos

- Após terminar, cada aluno **testa o sistema de outro colega**.
- Usa o **checklist de testes** para validar.
- Registra os resultados em um **Relatório de Incidente** (se encontrar falhas).

3. Checklist Simplificado - Testes

- [ ] Testes Unitários (cada função/método).
- [ ] Testes de Integração (módulos se comunicam corretamente).
- [ ] Testes Funcionais (o sistema faz o que promete).
- [ ] Testes de Performance (tempo de resposta aceitável).
- [ ] Testes de Segurança (login, permissões, injeções).
- [ ] Testes de Usabilidade (interface clara e acessível).
- [ ] Testes de Regressão (após alterações).

4 . Documentação

- [ ] Plano de Teste (objetivos, escopo, cronograma).
- [ ] Casos de Teste (entradas → resultados esperados).
- [ ] Relatório de Teste (resultados obtidos).
- [ ] Relatório de Incidente (falhas encontradas).
- [ ] Plano de Manutenção (corretiva, adaptativa, evolutiva).

5 - Checklist do Projeto (Django ou Laravel)

Ação	Status (Feito/Não feito)	Observações
Implementar autenticação (login/logout)	☐ feito	
Recuperação de senha	☐ feito	
Criar perfis de acesso (admin/usuário)	☐ feito	
Definir permissões por perfil	☐ feito	
CRUD completo de uma entidade principal	☐ feito	
Validação de dados nos formulários	☐ feito	
Dashboard inicial com visão geral	☐ feito	
Relatórios básicos (quantidade, status, métricas)	☐ feito	
Visualização em tabelas ou gráficos	☐ feito	
Plano de Teste documentado	☐ não feito	
Casos de Teste definidos	☐ feito	
Relatório de Teste preenchido	☐ feito	
Relatório de Incidente (se houver falhas)	☐ feito	
Plano de Manutenção (corretiva/adaptativa/evolutiva)	☐ feito	

6 - Modelo de Plano de Teste

ID do Teste	Funcionalidade	Entrada	Ação	Resultado Esperado	Resultado Obtido	Status (OK/Falha)
T01	Login	Usuário válido + senha correta	Submeter formulário	Acesso permitido	Obitido	OK
	Login	Usuário válido + senha incorreta	Submeter formulário	Mensagem de erro	Obitido	OK
T03	CRUD	Cadastro de novo cliente	Preencher dados e salvar	Cliente registrado no banco	Obitido	OK
T04	Relatório	Gerar relatório de reservas	Clicar em "Gerar"	Relatório exibido corretamente	Obitido	OK

7 - Modelo de Casos de Teste

Caso de Teste	Descrição	Pré-condições	Passos	Resultado Esperado
CT01	Testar login com credenciais válidas	Usuário cadastrado	1. Acessar tela de login 2. Inserir usuário e senha válidos 3. Clicar em "Entrar"	Sistema permite acesso
CT02	Testar exclusão de registro	Registro existente no banco	1. Acessar lista de registros 2. Selecionar registro 3. Clicar em "Excluir"	Registro removido da lista

8 - Modelo de Relatório de Teste

Funcionalidade Testada	Resultado Esperado	Resultado Obtido	Status	Observações
Login	Acesso permitido	Acesso permitido	OK	
Cadastro	Registro salvo	Erro de validação	Falha	Campo "email" não aceitou formato válido

9 - Modelo de Relatório de Incidente

ID do Incidente	Descrição da Falha	Impacto	Passos para Reproduzir	Solução Aplicada	Status
I01	Erro ao excluir registro	Alto – impede manutenção	1. Acessar lista 2. Selecionar registro 3. Clicar em "Excluir"	Corrigida query SQL	Resolvido
I02	Relatório não gera PDF	Médio – afeta exportação	1. Acessar dashboard 2. Clicar em "Exportar PDF"	Ajuste na biblioteca de exportação	Pendente

10 – Modelo de Plano de Manutenção

Tipo de Manutenção	Descrição	Exemplo no Projeto
Corretiva	Corrigir falhas identificadas nos testes	Ajustar bug no login
Adaptativa	Adequar o sistema a novos ambientes	Atualizar versão do framework
Evolutiva	Incluir novas funcionalidades	Adicionar exportação em Excel